

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
SPRING 2025**



**EXCALIBUR
LEARNLEAF ORGANIZER**

**EMMANUEL ASOMUGHA
IAN KELLY
ELLIOT MAI
BENJAMIN NICCUM
NICHOLAS PRATT**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	1.17.2025	EM	Document Creation
0.2	2.7.2025	EA, IK, EM, BN, NP	Complete Draft
0.3	2.10.2025	EA, IK, EM, BN, NP	Release Candidate 1

CONTENTS

1	Introduction	5
2	System Overview	5
3	Subsystem Definitions & Data Flow	6
3.1	Overall Subsystem Overview	6
3.2	Web (PWA) Subsystem	7
3.3	Mobile Subsystem	8
3.4	Firebase Subsystem	9
3.5	Summary of Data Flows	10
4	Web Subsystems	11
4.1	Web Operating System	11
4.2	Web Software Dependencies	11
4.3	Web Pages	12
4.4	LearnLeaf Functions	14
4.5	Index DB Utilities	15
4.6	IndexedDB	17
5	Mobile Subsystems	19
5.1	Mobile Hardware	19
5.2	Mobile Operating System	19
5.3	Mobile Software Dependencies	19
5.4	Mobile App Pages	20
5.5	LearnLeaf Functions	22
6	Firebase Subsystems	24
6.1	Firebase Operating System	24
6.2	Firebase Software Dependencies	24
6.3	Firebase Utilities	24
6.4	Firebase DB	25

LIST OF FIGURES

1	System-Wide Data Flow	7
2	Web Subsystem Overview	8
3	Mobile Subsystem Overview	9
4	Firebase Subsystem Overview	10
5	Progress Web Application Subsystem Overview	11
6	Web Pages Sublayer Overview	12
7	Web LearnLeaf Functions Sublayer Overview	14
8	Index DB Utilities Sublayer Overview	16
9	IndexedDB Utilities Sublayer Overview	17
10	Android Mobile Application Subsystem Overview	19
11	Mobile Pages Sublayer Overview	21
12	Mobile LearnLeaf Functions Sublayer Overview	22
13	Firebase Subsystem Overview	24
14	Firebase Utilities Sublayer Overview	24
15	Firebase DB Sublayer Overview	25

LIST OF TABLES

2	Summary of Data Flows Between Subsystems	10
---	--	----

1 INTRODUCTION

[4]

LearnLeaf Organizer is a task and project management platform designed to help students and educators efficiently organize their academic responsibilities. It provides tools for managing tasks, projects, and subjects, offering a structured and intuitive interface for tracking deadlines and priorities. The platform integrates various features, including a calendar view for scheduling, an archive for completed tasks, mass task import via spreadsheets, Learning Management System (LMS) connectivity for automatic synchronization, and automated email notifications for reminders.

This document serves as the Detailed Design Specification for the LearnLeaf Organizer, outlining the architectural components, system interactions, and technical specifications that define its functionality. It builds upon the requirements and high-level architecture defined in the Requirement Specification and Architectural Design documents. The following sections detail the system's structure, including its core layers, dependencies, and data flow.

2 SYSTEM OVERVIEW

[4]

The LearnLeaf Organizer system is structured into multiple layers, each handling a specific aspect of functionality. The core system consists of a Progressive Web Application (PWA) and an Android Mobile Application, both of which interact with a shared backend powered by Firebase. The PWA serves as the primary platform, enabling users to access task and project management tools from any web browser. The mobile application extends this functionality to Android devices, ensuring a seamless cross-platform experience.

The backend architecture is built on Firebase, which provides authentication, cloud storage, real-time synchronization, and automated processes such as email notifications and LMS integration. IndexedDB is utilized within the PWA to enable offline access, allowing users to interact with their data without an active internet connection. Changes made offline are synchronized with Firebase once connectivity is restored.

3 SUBSYSTEM DEFINITIONS & DATA FLOW

[2] [4]

The LearnLeaf Organizer system consists of multiple subsystems that interact through well-defined interfaces to facilitate task, project, and subject management. These subsystems handle frontend interactions, backend processing, authentication, data storage, synchronization, and offline caching. This section presents a detailed breakdown of subsystem interactions and the flow of data between them.

3.1 OVERALL SUBSYSTEM OVERVIEW

The system is structured into three primary layers: Web (PWA), Mobile, and Firebase. Each layer contains multiple subsystems that perform specific functions:

- **Web Layer (PWA):** Includes Web Pages (frontend) and three backend components: LearnLeaf Functions, Local DB Utilities, and IndexedDB. These backend elements manage user interactions, cache data locally, and sync with Firebase.
- **Mobile Layer:** Contains Mobile App Pages, which provide an interface similar to the Web Pages but optimized for mobile interaction. It also relies on LearnLeaf Functions for backend processing.
- **Firebase Layer:** Acts as the cloud backend, providing authentication, real-time data storage, synchronization, and automated processing through Firebase Utilities and the Firebase Database.

Figure 1 illustrates the overall data flow across these layers.

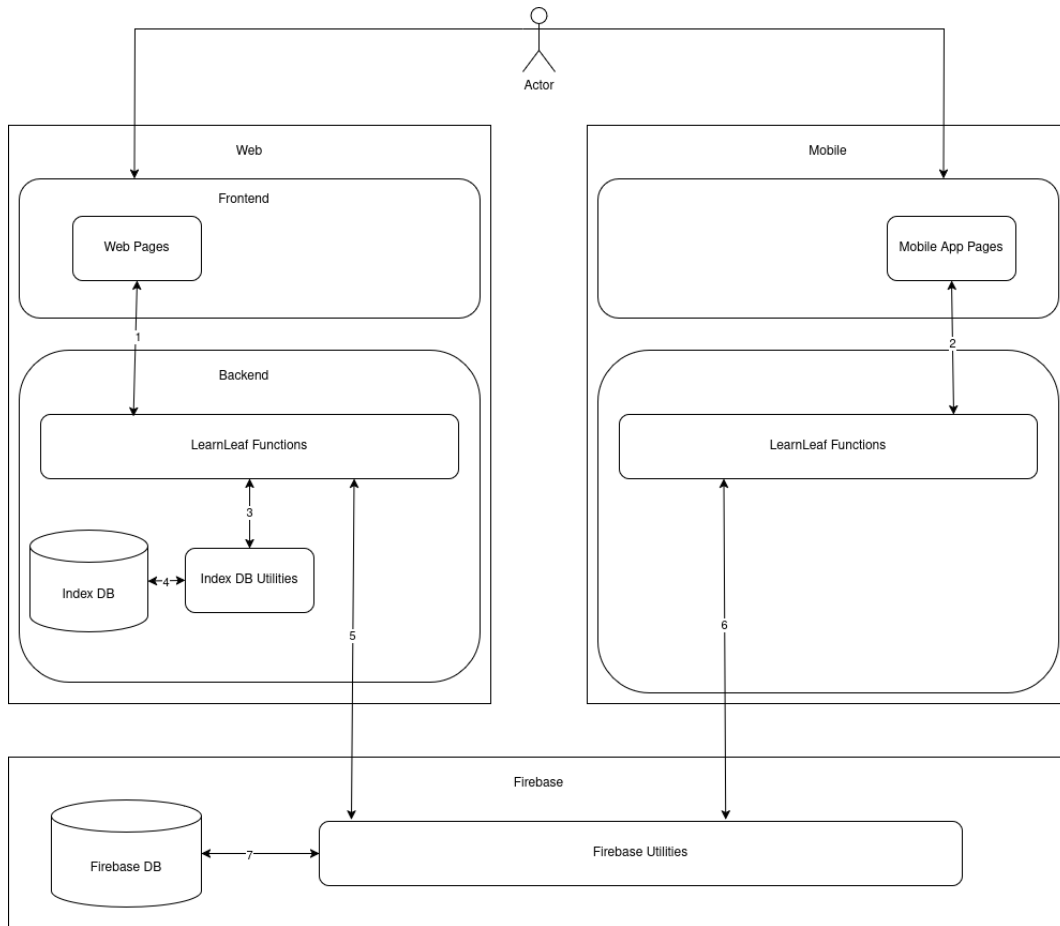


Figure 1: System-Wide Data Flow

3.2 WEB (PWA) SUBSYSTEM

The Web subsystem serves as the primary platform for user interaction via the Progressive Web Application. It consists of a frontend interface (Web Pages) and a structured backend composed of LearnLeaf Functions, Local DB Utilities, and IndexedDB.

- **Web Pages:** The user-facing component that renders tasks, projects, subjects, and calendar views. It captures user inputs and displays real-time updates.
- **LearnLeaf Functions:** Processes user requests, manages communication with Firebase, and interacts with Local DB Utilities.
- **Local DB Utilities:** Manages local storage interactions between LearnLeaf Functions and IndexedDB.
- **IndexedDB:** Stores cached data for offline use and syncs with Firebase when connectivity is restored.

Figure 2 provides a detailed look at the Web subsystem.

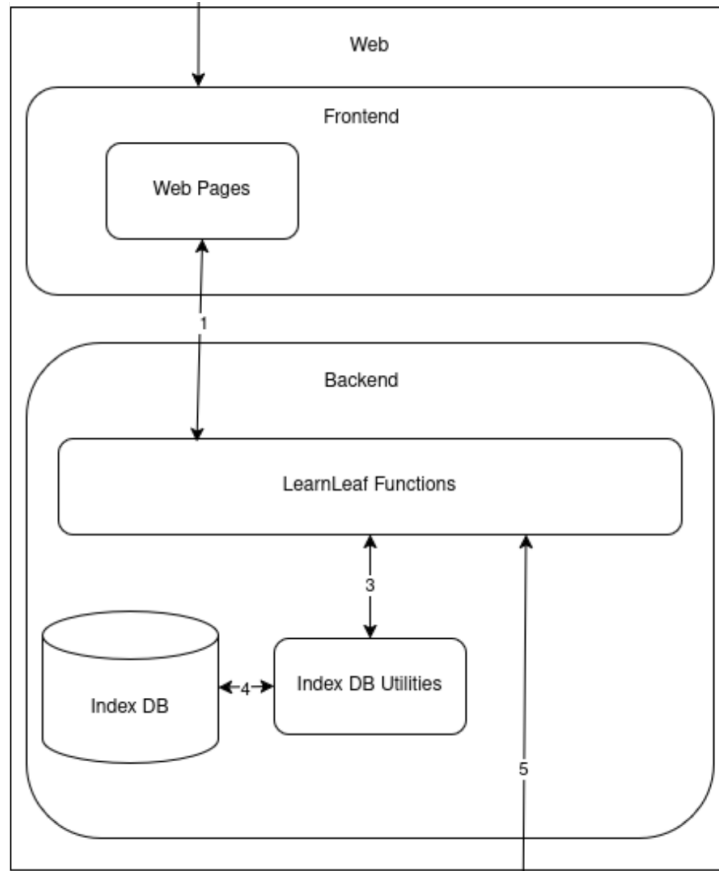


Figure 2: Web Subsystem Overview

3.2.1 DATA FLOW IN THE WEB SUBSYSTEM

Arrow 1: Web Pages to LearnLeaf Functions Web Pages send user-generated data such as task creation, updates, project management actions, and subject modifications to LearnLeaf Functions for processing. This ensures that all frontend modifications are properly validated and stored.

Arrow 3: LearnLeaf Functions to Local DB Utilities LearnLeaf Functions communicate with Local DB Utilities to cache data locally. This includes storing recent tasks, projects, and subjects for offline access.

Arrow 4: Local DB Utilities to IndexedDB Local DB Utilities facilitate the actual writing and retrieval of data from IndexedDB. IndexedDB maintains a structured offline cache of tasks, projects, and subjects.

Arrow 5: LearnLeaf Functions to Firebase Utilities LearnLeaf Functions ensure that all user-generated data is synchronized with Firebase Utilities. This enables real-time updates across devices.

3.3 MOBILE SUBSYSTEM

The Mobile subsystem provides a similar interface to the Web subsystem but is optimized for mobile devices. It relies on LearnLeaf Functions for processing and Firebase for cloud-based data storage.

- **Mobile App Pages:** The frontend application where users interact with tasks, projects, and subjects.
- **LearnLeaf Functions:** Handles data processing and communication with Firebase.

Figure 3 illustrates the Mobile subsystem and its interactions.

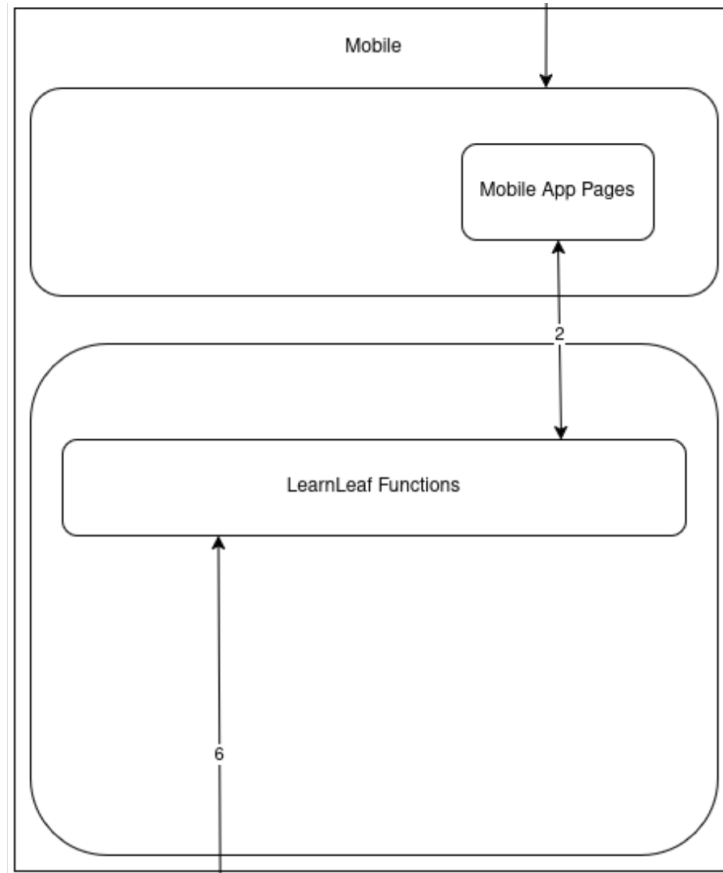


Figure 3: Mobile Subsystem Overview

3.3.1 DATA FLOW IN THE MOBILE SUBSYSTEM

Arrow 2: Mobile App Pages to LearnLeaf Functions Mobile App Pages send user interactions (task creation, updates, project edits) to LearnLeaf Functions for processing.

Arrow 6: LearnLeaf Functions to Firebase Utilities LearnLeaf Functions synchronize user data with Firebase Utilities to maintain consistency across devices.

3.4 FIREBASE SUBSYSTEM

Firebase serves as the backend for both the Web and Mobile subsystems, providing authentication, real-time synchronization, and cloud storage.

- **Firebase Utilities:** Manages authentication, background processing, and automated notifications.
- **Firebase Database:** Stores all user-related data, including tasks, projects, subjects, and calendar events.

Figure 4 details the Firebase subsystem.

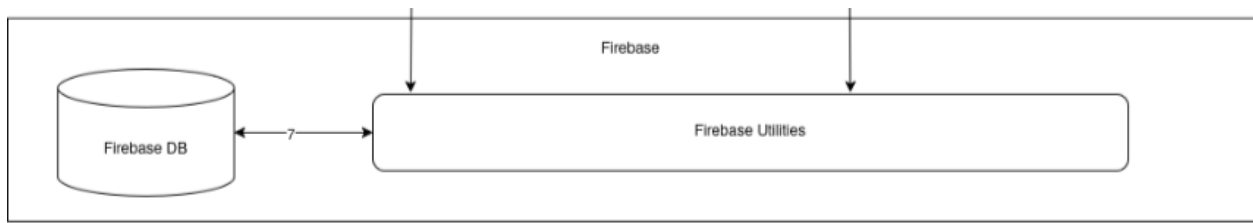


Figure 4: Firebase Subsystem Overview

3.4.1 DATA FLOW IN THE FIREBASE SUBSYSTEM

Arrow 7: Firebase Utilities to Firebase Database Firebase Utilities store and retrieve user data from the Firebase Database. This includes synchronizing tasks, projects, and subjects, as well as handling user authentication and automated notifications.

3.5 SUMMARY OF DATA FLOWS

Table 2 summarizes the flow of data between different subsystems.

No.	From	To
1	Web Pages	LearnLeaf Functions
2	Mobile App Pages	LearnLeaf Functions
3	LearnLeaf Functions (Web)	Local DB Utilities
4	Local DB Utilities	Index Database
5	LearnLeaf Functions (Web)	Firebase Utilities
6	LearnLeaf Functions (Mobile)	Firebase Utilities
7	Firebase Utilities	Firebase Database

Table 2: Summary of Data Flows Between Subsystems

4 WEB SUBSYSTEMS

[4]

The Progressive Web Application (PWA) serves as the central platform for managing tasks, projects, and subjects. It provides management interfaces for tasks, subjects, and projects, a calendar view for scheduling, an archive for completed items, mass task import via integrated spreadsheets, LMS connectivity for automatic task synchronization, automated email notifications, and user management. The PWA ensures seamless offline functionality and real-time synchronization with cloud storage, allowing users to maintain productivity regardless of their internet connection.

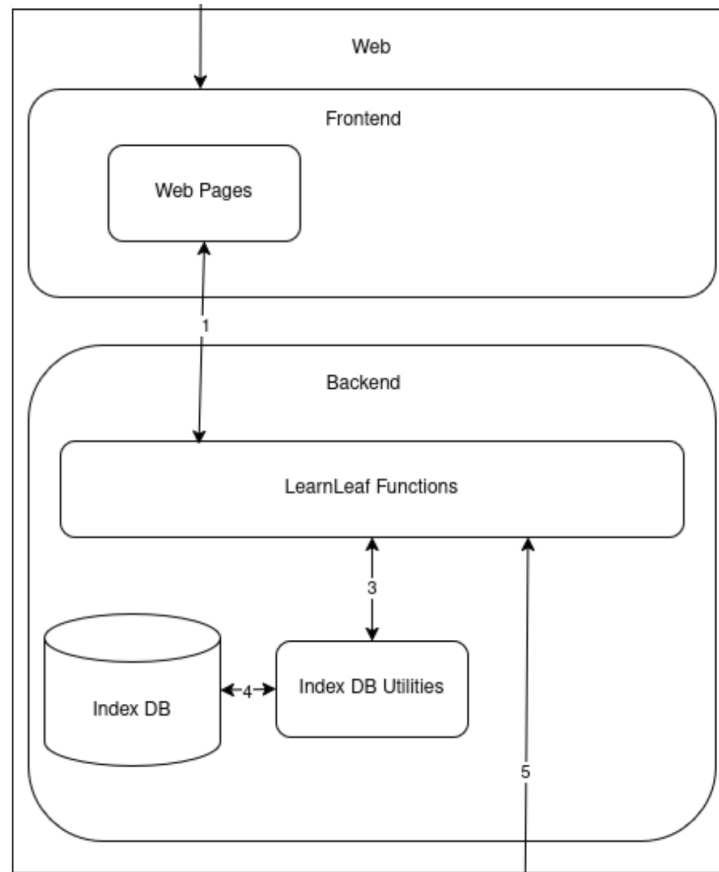


Figure 5: Progress Web Application Subsystem Overview

4.1 WEB OPERATING SYSTEM

The PWA runs in modern web browsers and is compatible with Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari. It is operating system agnostic, functioning smoothly on Windows, macOS, and Linux without requiring any installation beyond a web browser. Additionally, as a PWA, it can be installed on iOS and Android devices without needing to go through an app store, allowing users to access it like a native mobile application. The platform is optimized for Chromium-based browsers while ensuring compatibility across all modern web environments.

4.2 WEB SOFTWARE DEPENDENCIES

The PWA relies on various libraries and frameworks to support user interface rendering, state management, routing, data storage, email notifications, external integrations, and offline functionality.

CORE FRAMEWORK & ROUTING

- `react` (18.2.0) - Used to build the component-based user interface.
- `react-router-dom` (6.28.0) - Handles client-side navigation between different views.

DATABASE & STORAGE

- `firebase` (10.6.0) - Provides authentication and real-time database storage.
- `idb` (8.0.0) - Enables IndexedDB support for offline storage of user data.

4.3 WEB PAGES

The Web Pages subsystem provides the user interface where users interact with tasks, projects, and subjects. It is responsible for rendering dynamic content, capturing user inputs, and presenting real-time updates based on backend data. The interface includes multiple views such as the task dashboard, project dashboard, subject dashboard, calendar, and archives. It allows users to create, edit, delete, and organize their tasks while seamlessly integrating with other system components, including LMS synchronization and external calendar applications.

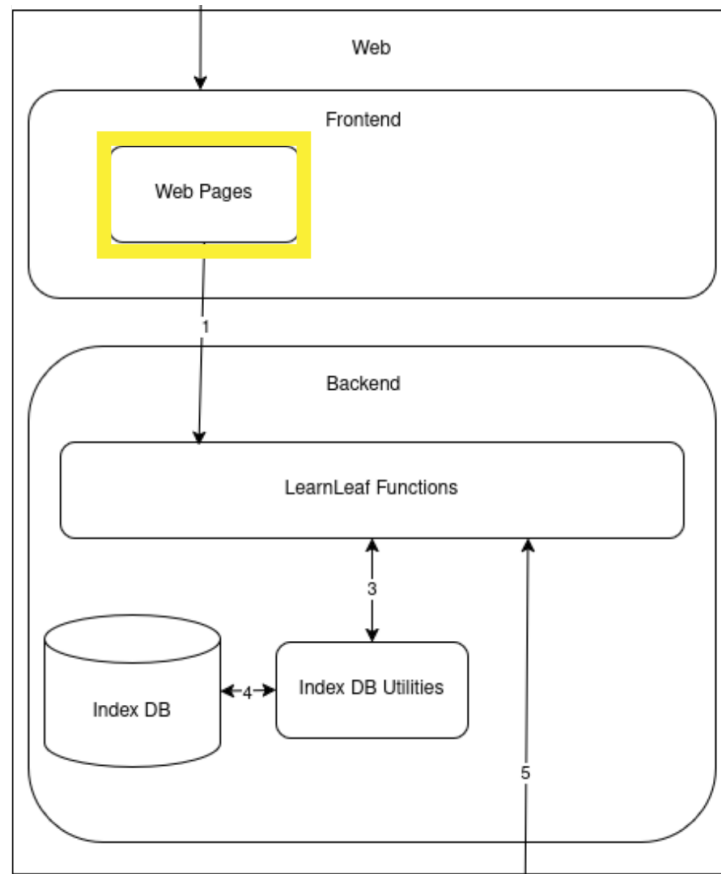


Figure 6: Web Pages Sublayer Overview

4.3.1 WEB PAGES OPERATING SYSTEM

The Web Pages subsystem runs within the browser, making it independent of any specific operating system. As part of the PWA, it is accessible on Windows, macOS, Linux, iOS, and Android.

4.3.2 WEB PAGES SOFTWARE DEPENDENCIES

The Web Pages subsystem depends on frontend libraries that enable efficient rendering and navigation.

ROUTING

- `react-router-dom` - Manages navigation between different views.

UI & STYLING

- `@mui/material` (6.1.6) - Provides Material UI components for styling and layout.
- `react-color` (2.19.3) - Used for color selection when assigning colors to subjects.

STATE MANAGEMENT & DATA HANDLING

- `@tanstack/react-query` (5.59.16) - Manages client-server data synchronization for tasks, projects, and subjects.
- `lodash.debounce` (4.0.8) - Improves performance by reducing redundant function executions.

TASK & DATA MANAGEMENT

- `react-big-calendar` (1.11.6) - Displays tasks in a calendar format for better scheduling.
- `ical.js` (2.1.0) - Enables iCalendar (.ics) integration for syncing external calendars.

SPREADSHEETS & DATA GRIDS (FOR MASS TASK IMPORT)

- `@handsontable/react` (14.6.1) - Provides a spreadsheet interface for mass task imports.
- `react-data-grid` (7.0.0-beta.47) - Used for rendering and manipulating large sets of tabular data.

4.3.3 WEB PAGES PROGRAMMING LANGUAGES

The Web Pages subsystem is developed using JavaScript (ES6+) for defining application logic, JSX for structuring UI components, and CSS for styling and responsive design.

4.3.4 WEB PAGES DATA STRUCTURES

The Web Pages subsystem primarily manages three types of data objects: tasks, projects, and subjects. The Task object contains details such as name, due date, priority, and project or subject linkage. The Project object stores project details, including associated subjects and related tasks. The Subject object includes metadata such as the subject name, semester, and color coding.

4.3.5 WEB PAGES DATA PROCESSING

The Web Pages subsystem processes data by implementing sorting algorithms to arrange tasks based on due date, priority, and status. It uses React state management to dynamically update the interface as users interact with the system. Event handling mechanisms ensure that changes are reflected immediately without requiring manual refreshes.

4.4 LEARNLEAF FUNCTIONS

LearnLeaf Functions serve as the backend logic layer, managing authentication, task synchronization, LMS integration, and email notifications. These functions interact between the frontend Web Pages and data storage systems, ensuring that all data is updated in real time.

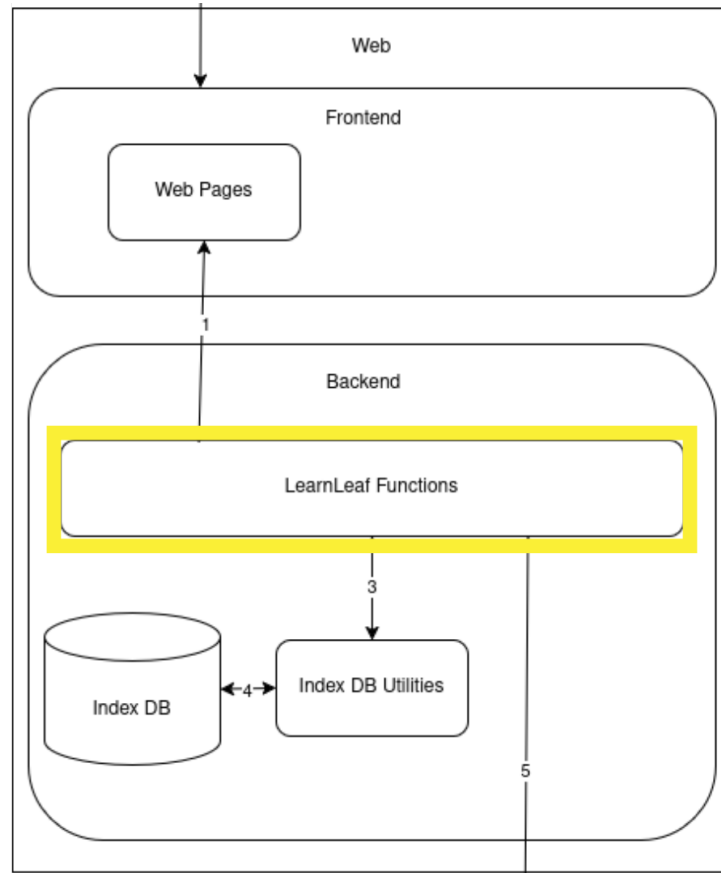


Figure 7: Web LearnLeaf Functions Sublayer Overview

4.4.1 LEARNLEAF FUNCTIONS OPERATING SYSTEM

LearnLeaf Functions operate in a serverless cloud environment using Firebase Functions. They do not require a dedicated operating system, as they are executed on-demand within the Firebase ecosystem.

4.4.2 LEARNLEAF FUNCTIONS SOFTWARE DEPENDENCIES

LearnLeaf Functions require backend libraries for executing server-side logic and interacting with external services.

- `firebase-functions` - Runs cloud-based serverless functions for authentication, task updates, and LMS integrations.
- `firebase-admin` - Provides administrative access to Firestore.
- `@sendgrid/mail` - Sends automated email notifications to users.

AUTOMATED EMAIL NOTIFICATIONS & EXTERNAL COMMUNICATION

- `@sendgrid/mail` (8.1.3) - Sends automated email reminders for upcoming tasks.
- `firebase-functions` - Runs serverless functions to trigger email notifications.
- `node-fetch` (3.3.2) - Fetches data from external Learning Management Systems (LMSs).

4.4.3 LEARNLEAF FUNCTIONS PROGRAMMING LANGUAGES

LearnLeaf Functions are written in JavaScript and executed in a Node.js runtime environment.

4.4.4 LEARNLEAF FUNCTIONS DATA STRUCTURES

LearnLeaf Functions process data stored in Firestore collections, including user authentication details, task data, project information, and subject metadata. When fetching data from LMS integrations, functions map external assignments to the internal task structure before storing them in Firestore.

4.4.5 LEARNLEAF FUNCTIONS DATA PROCESSING

Data processing in LearnLeaf Functions includes checking task deadlines and triggering automated email notifications. Functions also fetch and synchronize LMS assignments by retrieving external API data, mapping the assignments to Firestore structures, and ensuring consistency with user data.

4.5 INDEX DB UTILITIES

The Index DB Utilities manage offline storage and synchronization between IndexedDB and Firestore. This subsystem ensures that users can access and modify their data even when offline, with changes being synchronized once an internet connection is available.

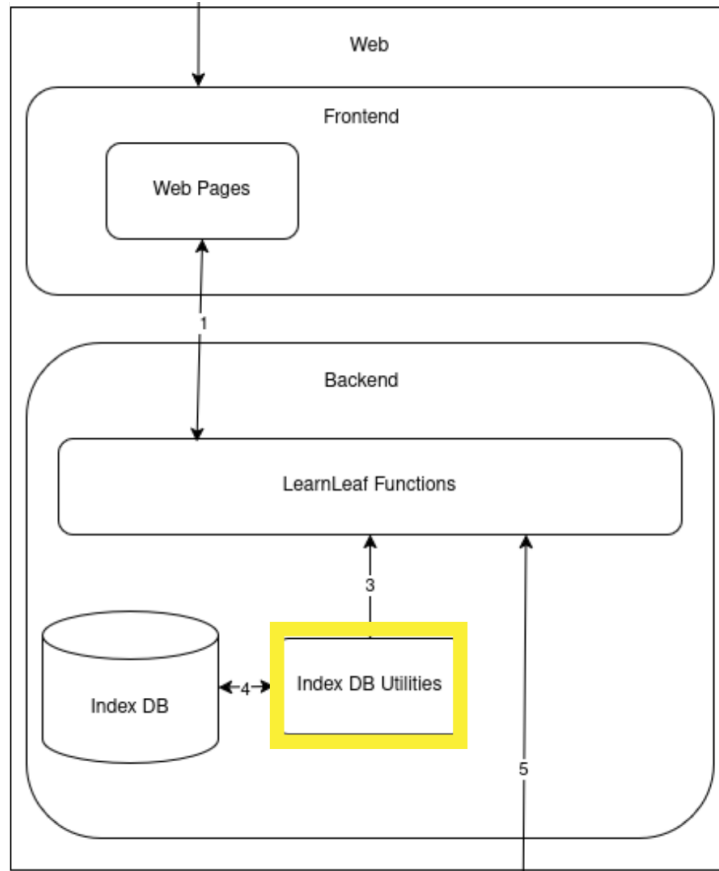


Figure 8: Index DB Utilities Sublayer Overview

4.5.1 INDEX DB UTILITIES OPERATING SYSTEM

The Index DB Utilities subsystem runs within the browser and does not require a specific operating system. It leverages IndexedDB for local storage, making it fully functional on Windows, macOS, Linux, iOS, and Android.

4.5.2 INDEX DB UTILITIES SOFTWARE DEPENDENCIES

The Index DB Utilities subsystem relies on IndexedDB to store and manage offline data.

- `idb` - Provides an abstraction layer for IndexedDB, making it easier to store and retrieve offline data.

4.5.3 INDEX DB UTILITIES PROGRAMMING LANGUAGES

The Index DB Utilities subsystem is implemented in JavaScript and operates as a set of background functions that handle data retrieval, storage, and synchronization.

4.5.4 INDEX DB UTILITIES DATA STRUCTURES

IndexedDB maintains structured object stores for tasks, projects, and subjects. Each store is indexed by a unique identifier, ensuring fast retrieval and efficient local data management.

4.5.5 INDEX DB UTILITIES DATA PROCESSING

The Index DB Utilities subsystem initializes the database upon first use, setting up object stores for tasks, projects, and subjects. It retrieves and caches data for offline access and synchronizes local changes with

Firestore when an internet connection is restored. Synchronization logic ensures that conflicts between local and remote changes are resolved efficiently.

4.6 INDEXEDDB

IndexedDB serves as the local storage system for the application, providing offline access to tasks, projects, and subjects. It enables users to interact with their data even without an internet connection and automatically synchronizes changes with Firebase when connectivity is restored. IndexedDB improves performance by reducing reliance on cloud storage for frequent read and write operations.

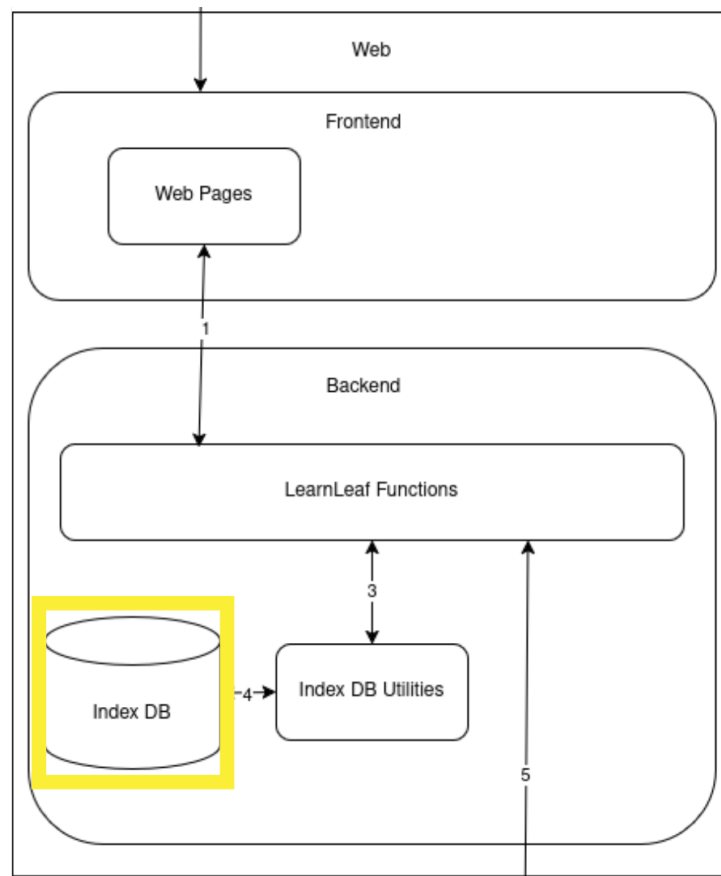


Figure 9: IndexedDB Utilities Sublayer Overview

4.6.1 INDEXEDDB OPERATING SYSTEM

IndexedDB is a browser-based database that operates independently of the operating system. It is natively supported by modern browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari, making it accessible on Windows, macOS, Linux, iOS, and Android without additional installation.

4.6.2 INDEXEDDB SOFTWARE DEPENDENCIES

IndexedDB is a built-in browser database that does not require external installation. The application utilizes the `idb` library to simplify database interactions, providing a higher-level API for managing local data storage.

4.6.3 INDEXEDDB PROGRAMMING LANGUAGES

IndexedDB is accessed using JavaScript via browser APIs. The `idb` library provides an abstraction layer to facilitate interactions with IndexedDB, making data retrieval and storage more efficient.

4.6.4 INDEXEDDB DATA STRUCTURES

IndexedDB organizes data into object stores rather than traditional tables. The application maintains separate stores for tasks, projects, and subjects, each identified by a unique key. The structure mirrors Firestore collections to enable seamless synchronization between local storage and the cloud.

4.6.5 INDEXEDDB DATA PROCESSING

IndexedDB enables efficient data caching by storing frequently accessed user data locally. It supports asynchronous transactions to improve performance and prevent UI blocking. Data synchronization occurs when an internet connection is available, ensuring consistency between local storage and Firestore. IndexedDB also allows bulk operations, optimizing task and project imports from spreadsheets.

5 MOBILE SUBSYSTEMS

[1]

The Android Mobile Application is a companion platform to the web-based Progressive Web Application (PWA), offering core functionalities for task, project, and subject management. It provides users with a native mobile experience while maintaining data synchronization with the main application through Firebase integration.

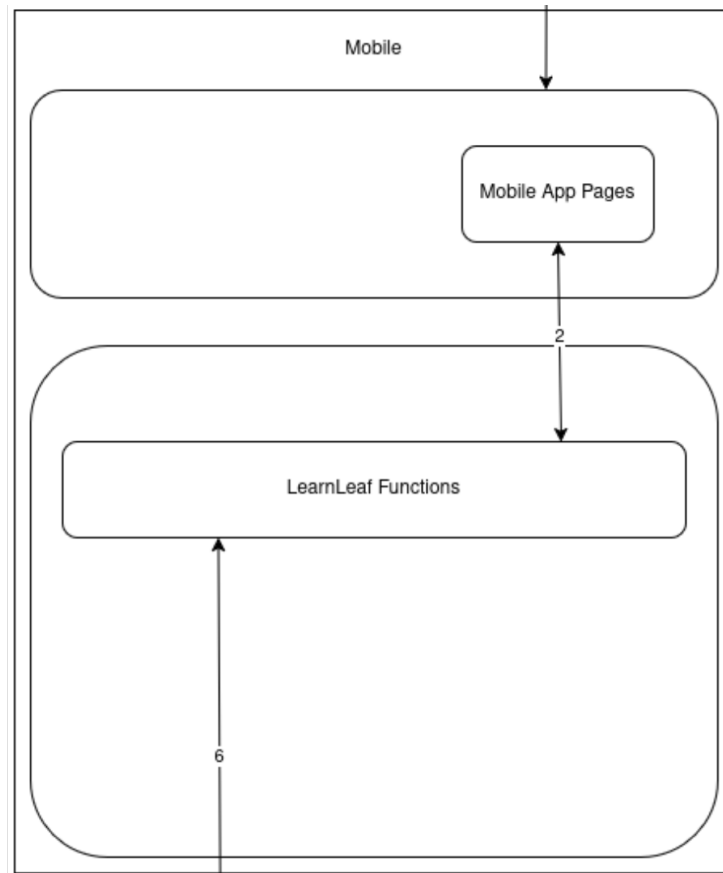


Figure 10: Android Mobile Application Subsystem Overview

5.1 MOBILE HARDWARE

The application is designed to run on Android-compatible smartphones and tablets. It requires no specific hardware components beyond those typically found in modern Android devices, such as a touch-screen for user input and sufficient storage for app installation and data caching.

5.2 MOBILE OPERATING SYSTEM

The application is developed for the Android operating system. It is compatible with Android versions from 5.0 (Lollipop) and above, with a specific API level targeted at 34.

5.3 MOBILE SOFTWARE DEPENDENCIES

- Android Studio (primary IDE)
- Android SDK (compileSdk 34 and targetSdk 34)

- Android Gradle Plugin 8.6.1
- Google Services Plugin 4.4.2
- AndroidX Libraries:
 - AppCompat 1.7.0
 - ConstraintLayout 2.1.4
 - Activity 1.9.2
 - Fragment 1.8.4
- Google Material Design 1.12.0
- Material Icons Extended 1.5.0
- JUnit 4.13.2 and JUnit Jupiter 5.8.2
- Espresso 3.6.1
- TestNG 6.9.6
- Google Play Services Analytics 18.1.0
- Firebase SDK:
 - Firebase BOM (Bill of Materials) 33.4.0
 - Firebase Authentication 23.0.0
 - Firebase Analytics
 - Firebase Firestore 25.1.0

5.4 MOBILE APP PAGES

The mobile app pages represent the user interface and functionality of the Android application, providing access to task, project, and subject management features.

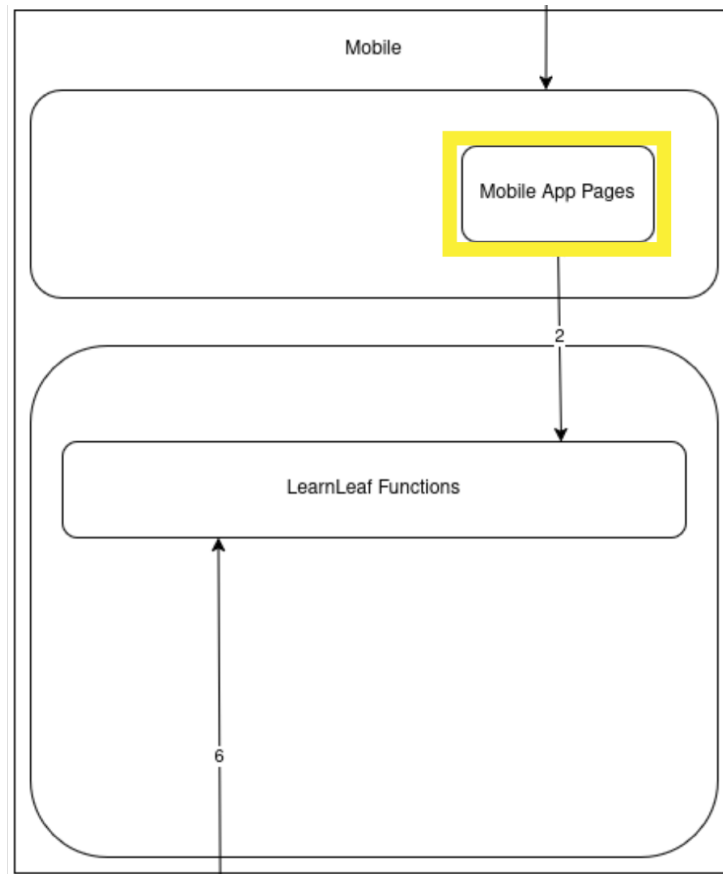


Figure 11: Mobile Pages Sublayer Overview

5.4.1 MOBILE APP PAGES OPERATING SYSTEM

The pages are designed to run on the Android operating system, compatible with versions 5.0 (Lollipop) and above.

5.4.2 MOBILE APP PAGES SOFTWARE DEPENDENCIES

The pages utilize the same software dependencies as the main Android Mobile Application, including AndroidX libraries, Google Material Design, and other UI-related components.

5.4.3 MOBILE APP PAGES PROGRAMMING LANGUAGES

- Java: Used for implementing the application logic and functionality
- XML: Used for defining the user interface layouts and resources

5.4.4 MOBILE APP PAGES DATA STRUCTURES

- Task: Represents individual tasks with properties such as title, description, due date, priority, and associated subject or project
- Project: Stores project information, including title, description, associated tasks, and linked subjects
- Subject: Contains subject details like name, color coding, and related tasks or projects
- User: Manages user profile information and authentication details

5.4.5 MOBILE APP PAGES DATA PROCESSING

- CRUD Operations: Implements Create, Read, Update, and Delete operations for tasks, projects, and subjects
- Offline Caching: Implements local caching of data to allow for offline access and operations
- Data Filtering and Sorting: Implements algorithms to filter and sort tasks, projects, and subjects based on various criteria such as due date, priority, or alphabetical order

5.5 LEARNLEAF FUNCTIONS

LearnLeaf Functions are custom functions that implement Firebase utilities to interact with Firebase Firestore database, providing real-time data synchronization, user authentication, and analytics for the Android Mobile Application.

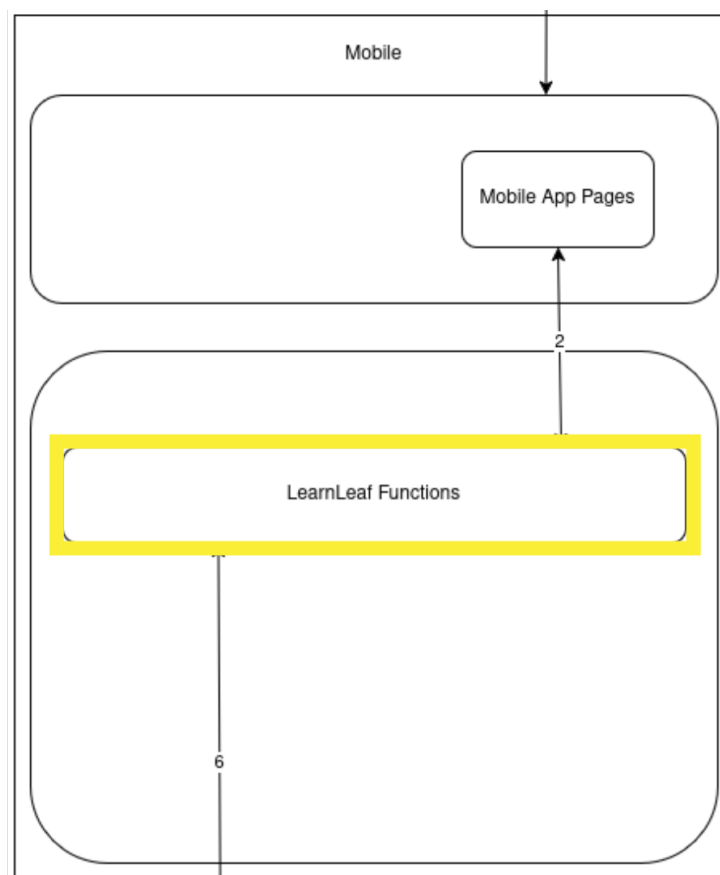


Figure 12: Mobile LearnLeaf Functions Sublayer Overview

5.5.1 LEARNLEAF FUNCTIONS OPERATING SYSTEM

LearnLeaf Functions are not tied to a specific operating system as they interact with the cloud-based Firebase services.

5.5.2 LEARNLEAF FUNCTIONS SOFTWARE DEPENDENCIES

- Firebase Admin SDK
- Firebase Functions SDK

5.5.3 LEARNLEAF FUNCTIONS PROGRAMMING LANGUAGES

LearnLeaf Functions for the mobile app are written entirely in Java.

5.5.4 LEARNLEAF FUNCTIONS DATA STRUCTURES

LearnLeaf Functions interact with Firebase Firestore, which stores data for tasks, projects, subjects, and users in a NoSQL database structure. Each entity is represented as a document within a collection, allowing for flexible and scalable data storage.

5.5.5 LEARNLEAF FUNCTIONS DATA PROCESSING

- **CRUD Operations:** Implements Create, Read, Update, and Delete operations for tasks, projects, and subjects using Firebase Firestore
- **Data Synchronization:** Utilizes Firebase Firestore's real-time synchronization capabilities to ensure data consistency between the mobile app and the web application
- **User Authentication:** Processes user login and registration through Firebase Authentication services
- **Analytics:** Tracks user engagement and app performance using Firebase Analytics

6 FIREBASE SUBSYSTEMS

[3] [4]

Firebase provides a centralized backend for authentication, cloud storage, real-time synchronization, and automated processes, making it ideal for cross-device implementation between the Android App and PWA versions. It ensures seamless data access, secure authentication, and efficient background processing across platforms.

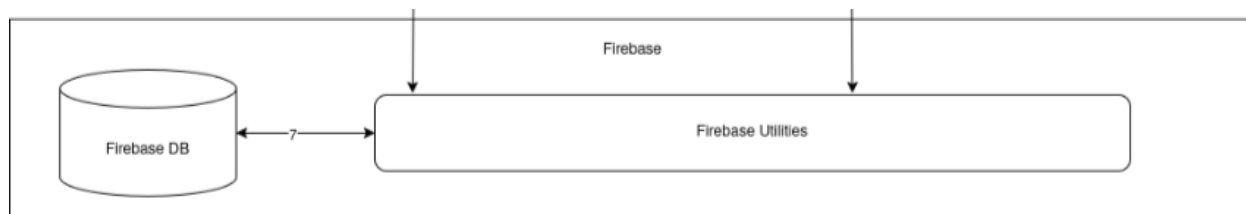


Figure 13: Firebase Subsystem Overview

6.1 FIREBASE OPERATING SYSTEM

Firebase is a cloud-based backend service that operates on Google Cloud Platform. It does not require any specific operating system for deployment or execution. The application interacts with Firebase through web APIs, making it accessible from any device with an internet connection, regardless of the operating system.

6.2 FIREBASE SOFTWARE DEPENDENCIES

Firebase provides a suite of backend services, including authentication, real-time database, cloud storage, and serverless functions. The system relies on Firebase SDKs and additional libraries for seamless integration, communication with Firestore, and executing automated backend tasks.

6.3 FIREBASE UTILITIES

Firebase Utilities are built-in backend services provided by Firebase, handling authentication, data retrieval, task synchronization, and email notifications. These utilities enable seamless interaction with Firestore, manage user sessions, and trigger cloud functions for automation. By leveraging these pre-configured services, the system benefits from real-time data updates and external integrations with minimal custom backend development.

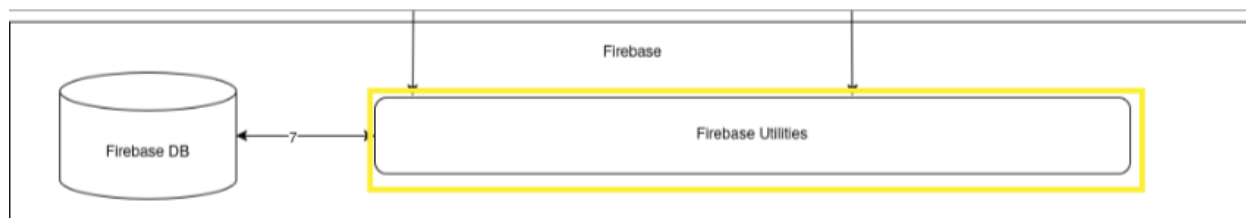


Figure 14: Firebase Utilities Sublayer Overview

6.3.1 FIREBASE UTILITIES OPERATING SYSTEM

Firebase Utilities operate within Firebase Functions, running in a serverless cloud environment. Since they are executed remotely on Google Cloud, they are independent of any local operating system.

6.3.2 FIREBASE UTILITIES SOFTWARE DEPENDENCIES

Firebase Utilities rely on Firebase Authentication for user management, Firestore for structured data storage, and Firebase Functions for executing automated backend operations. Additional integrations include external APIs for retrieving LMS data.

6.3.3 FIREBASE UTILITIES PROGRAMMING LANGUAGES

Firebase Utilities are pre-configured Firebase services that interact with applications via JavaScript APIs. Firebase Functions supporting these utilities are written in JavaScript and executed in a Node.js runtime.

6.3.4 FIREBASE UTILITIES DATA STRUCTURES

Firestore, used by Firebase Utilities, organizes data in collections and documents. User authentication data, task lists, project information, and subject metadata are stored in Firestore, structured in a way that supports efficient retrieval and real-time synchronization.

6.4 FIREBASE DB

Firebase DB serves as the cloud-based storage system for the application, utilizing Firestore, a NoSQL database, to maintain structured data. It ensures high availability, scalability, and real-time synchronization across devices, enabling users to access up-to-date information on tasks, projects, and subjects.

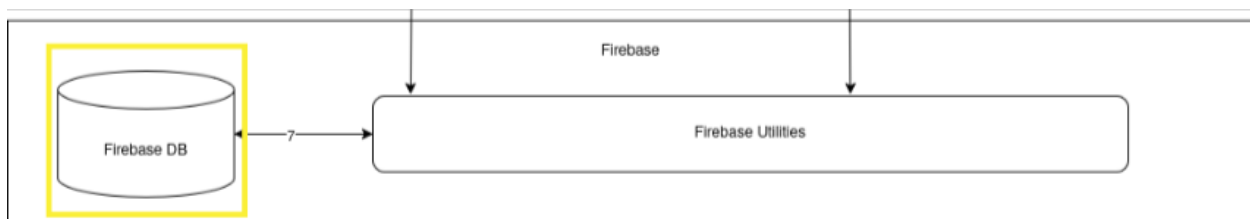


Figure 15: Firebase DB Sublayer Overview

6.4.1 FIREBASE DB OPERATING SYSTEM

Firebase DB operates entirely in the cloud and does not require an on-premise operating system. It is hosted and managed on Google Cloud Platform, providing a serverless environment that scales automatically based on demand.

6.4.2 FIREBASE DB SOFTWARE DEPENDENCIES

Firestore, Firebase's NoSQL cloud database, is the primary data storage service for the system. Firebase Authentication is used for access control, ensuring that only authorized users can modify or retrieve specific data. Firebase Functions support automated background processes related to data storage and retrieval.

6.4.3 FIREBASE DB PROGRAMMING LANGUAGES

Firestore is accessed using JavaScript through Firebase SDKs. Queries and data manipulations are performed via Firestore's API, with Firebase Functions handling background operations.

6.4.4 FIREBASE DB DATA STRUCTURES

Firestore stores data in a hierarchical structure consisting of collections and documents. Collections include users, tasks, projects, and subjects, with each document storing structured JSON-like data optimized for retrieval and synchronization.

6.4.5 FIREBASE DB DATA PROCESSING

Firestore supports real-time synchronization, automatic indexing for optimized queries, and background triggers for updates. Firestore security rules enforce access control policies, while Firebase Functions execute scheduled tasks such as email reminders and LMS data imports.

REFERENCES

- [1] Perplexity AI. Perplexity ai, 2023. AI-powered search engine.
- [2] diagrams.net. draw.io - Online Diagram Software, 2024.
- [3] Google LLC. Firebase: A comprehensive app development platform, 2023. Cloud-based app development platform.
- [4] OpenAI. Chatgpt, 2023. Large language model.